Boundary-Oriented Evaluation of a Deployed AI Agent Service

A Reproducible Public-Surface Case Study

Shafaet Brady Hussain | Independent researcher, TalkToAI Research, United Kingdom | 26 July 2026

Abstract

AI agent services combine web applications, authentication, model routing, provider integrations, and command-line clients. A model-only benchmark cannot establish whether such a service enforces its public security boundary or whether its advertised routes behave consistently. This paper reports a reproducible, boundary-oriented evaluation of the public ZeroThink service. The study separates live public-surface behaviour from static inspection of a local source snapshot, and intentionally excludes account cookies, production secrets, real provider keys, and authentication bypass attempts. The 9 July 2026 run executed 23 predefined checks: six public-route checks, six command-line API checks, four agent API checks, and seven static integration checks. All predefined checks passed in that snapshot, and the response scan found no token-shaped secret patterns. The result is evidence that the tested boundary behaved as specified at that time; it is not evidence of model intelligence, cryptographic security, resistance to a determined adversary, or current production status. We release the test protocol, scored outputs, source hashes, and this manuscript to make the claims independently auditable and readily falsifiable by future runs.

Keywords: AI agents; security testing; reproducibility; authentication boundaries; systems evaluation; web services

1. Introduction

Deployed AI systems are services, not only model weights. Their practical behaviour depends on web routes, API guards, authentication, routing code, configuration, and integration boundaries. This makes it easy for a project to make a broad system claim while supplying only a model demonstration, or conversely to report a model score that says little about the safety of a public service.

This study presents a narrowly scoped case study of the ZeroThink public service at https://zerothink.talktoai.org. The goal is not to demonstrate superior language-model quality. It is to test whether a small, predeclared set of public routes and guard behaviours matched their intended observable contracts, without using privileged access. This boundary-oriented approach follows the general principle that security-relevant functionality should be tested against explicit, repeatable expectations rather than inferred from marketing copy or a successful login flow [1,2].

2. Research questions

RQ1: Did selected public pages return the expected status and content markers during the run? RQ2: Did unauthenticated CLI and agent endpoints enforce their documented input and authentication boundaries? RQ3: Did a local source snapshot contain the integration and validation markers expected by the protocol? RQ4: Did the captured public responses contain selected token-shaped secret patterns? These questions intentionally exclude claims about model reasoning quality, penetration resistance, availability under load, privacy-law compliance, cryptographic correctness, or the security of third-party providers.

3. Materials and method

The live target was the public ZeroThink deployment. The protocol used no user cookies, no real provider API keys, no private server credentials, no customer data, and no attempts to obtain unauthorized access. Two local direct-mode identity/capability branches were exercised with a deliberately fake placeholder key only where the implementation returned before an external provider call. The evaluation did not send model-generation requests through a paid provider lane.

The static component used a local source snapshot and recorded SHA-256 digests for the inspected files. Static results must therefore not be read as proof that the same revision was deployed live. The live and static layers are reported separately throughout.

Table 1. Predeclared checks and expected observable contracts.

Layer	Checks	Expected contract
Public pages	6	HTTP 200, expected markers, no selected token-shaped secret patterns
CLI API	6	Input errors and unauthorized calls are rejected; device login remains pending before approval
Agent API	4	Unauthorized generation is blocked; safe local identity/capability branches return expected markers
Static source snapshot	7	Expected routing, validation, Paper Creator, and CLI markers are present

4. Results

The 9 July 2026 snapshot passed all 23 predefined checks. The response scan reported zero rows containing the selected token-shaped patterns. The mean latency across benchmark rows with latency was 0.134 seconds; this is descriptive only and not a load, reliability, or service-level measurement.

Six public routes were checked: the root page, Research Paper Creator, research page, FAQ, documentation page, and CLI connect route. Every route returned HTTP 200 in three requests and contained its expected marker. The route-level means ranged from 0.054 seconds for the CLI-connect route to 1.096 seconds for the FAQ route. No claim about general throughput or uptime follows from these 18 requests.

The CLI endpoint returned errors for a missing action and a missing device code. It rejected an unauthenticated identity request and an unsupported protected action. A device-start request returned a success response with the expected device-login fields; a subsequent unapproved poll returned authorization_pending with HTTP 202. The agent endpoint rejected unauthenticated OpenZero generation and an unsupported OpenZero direct-mode attempt with HTTP 403. The two safe local probes returned the expected identity/capability markers.

Table 2. Scored outcomes.

Area	Passed / total	Interpretation
Public pages	6 / 6	Selected routes returned expected markers in three requests each.
CLI API	6 / 6	Guard, device-start, and pending-state contracts matched the test.
Agent API	4 / 4	Tested unauthorized requests were blocked; local branches returned expected markers.
Static snapshot	7 / 7	Selected implementation markers were present locally.
Secret-pattern scan	0 hits	No selected token-shaped pattern appeared in captured responses.

5. Discussion

The central result is architectural rather than cognitive. A public AI service can be evaluated as a set of observable security and reliability contracts: whether protected routes fail closed, whether a device-flow remains pending before approval, whether unauthenticated generation is blocked, and whether a disclosed source snapshot includes the claimed interface markers. Such checks complement—rather than replace—model evaluation. Model quality, safety alignment, cost, and reliability need distinct protocols [3–5].

The study also illustrates the value of reporting negative and limiting results. Earlier local-model measurements in the accompanying release showed that a direct local model path and a wrapped service path may behave differently. This paper therefore makes no claim that ZeroThink improves underlying model quality or that its local model lane matches any cloud provider.

6. Limitations

This is a small case study, not a certification or a security audit. The live checks were performed on 9 July 2026 and can become stale after any deployment. The local source snapshot was not cryptographically tied to the live deployment. Three requests per public page do not measure availability, load tolerance, or geographic performance. Pattern matching cannot prove that no sensitive information was exposed. The test suite does not assess vulnerabilities outside the chosen requests, authorization bypasses, cross-site attacks, dependencies, or infrastructure configuration. No standard reasoning benchmark or human-subject evaluation was conducted.

7. Conclusion

This paper documents a dated, evidence-bounded public-surface evaluation of a deployed AI agent service. In the recorded 9 July 2026 snapshot, all 23 predefined checks passed and the selected response scan found no token-shaped secret patterns. The result should be used as a reproducible baseline and a regression target, not as a broad quality, security, or intelligence claim.

Data and code availability

The release package contains the test runner, scored CSV, summary JSON, and a public source ledger at https://github.com/ResearchForumOnline/research. The public site directory is https://research.talktoai.org. The source snapshot hashes and full result matrix are included in data/benchmarks/zerothink-system-benchmark-2026-07-09.json.

Conflict of interest

The author builds and maintains components in the TalkToAI ecosystem, including the service evaluated here. This was a self-evaluation. The manuscript therefore avoids claims of independent audit, certification, or model superiority and publishes the protocol/results for external scrutiny.

AI-use disclosure

AI-assisted drafting tools were used for editorial structuring and language refinement. The study design, source artifacts, recorded results, limitations, and final claims were reviewed against the released evidence. No generated text is presented as an experimental observation without a corresponding artifact.

References

1. National Institute of Standards and Technology. Technical Guide to Information Security Testing and Assessment (SP 800-115). 2008. https://doi.org/10.6028/NIST.SP.800-115

2. OWASP Foundation. Application Security Verification Standard 5.0.0. 2025. https://owasp.org/www-project-application-security-verification-standard/

3. Sculley D, et al. Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems 28. 2015.

4. Amershi S, et al. Software Engineering for Machine Learning: A Case Study. ICSE-SEIP. 2019. https://doi.org/10.1109/ICSE-SEIP.2019.00042

5. Mitchell M, et al. Model Cards for Model Reporting. FAT* 2019. https://doi.org/10.1145/3287560.3287596